

Measured Forgetting:

Context Management for Agentic Local LLMs

Barak Achillah Asidi
CIF AI
thexi.dev
research@thexi.dev

May 2026

Abstract

We present *measured forgetting*, a context management method for local language models performing multi-round tool-calling on consumer hardware. Unlike cloud-hosted APIs with 128K+ context windows, local models on Apple Silicon (8B–14B parameters, 8K–16K context) exhaust their context within 3–4 tool rounds — each round appends both a model response and a tool result. Existing approaches either impose a fixed round limit (truncating investigation prematurely) or rely on the user to manage context (shifting cognitive load away from the task).

Measured forgetting treats context as a finite resource to be managed *during* inference, not before it. The method monitors context utilisation in real-time, partitions the message history into three zones (pinned, compressible, recent), and invokes the model itself to compress the compressible zone into a summary — all within the same generation loop. Version 2 extends uniform compression with three novel instruments — an Influence Equation (multi-dimensional scoring with super-linear $\Phi(B) = B^{1.5}$ resonance), Trace Topology (causal chain preservation), and a Problem Taxonomy (task-class-adaptive compression strategy). Across 7 models from 5 independent families (Alibaba, Mistral, Meta, Google, Microsoft), V2 outperforms V1 on every model tested, with improvements ranging from +0.15 to +0.46 (out of 3.0), and the largest gains on the weakest summarisers.

Combined with a KV-cache-persistent session worker that eliminates redundant prompt re-processing across rounds, the approach enables unbounded tool-calling depth on a 16K context window while maintaining response quality. Both techniques are deployed in production: measured forgetting in a Tauri desktop application (B.app, Qwen3 8B, M4 Mac), and a variant of the compaction protocol in a cloud-hosted chat orchestrator. Code and benchmark harness: <https://github.com/cif-ai/measured-forgetting>.

Keywords: context management, local inference, tool-calling agents, KV cache, information theory, influence scoring, problem taxonomy, llama.cpp

1 The Problem

1.1 Agentic loops exhaust context fast

A local LLM acting as an agent follows a pattern: the user asks a question, the model generates a response containing a tool call, the orchestrator executes the tool, injects the result, and the model generates again. Each round appends two messages (assistant response + tool result), typically 200–800 tokens each. On a 16K context window with a 2K system prompt, the usable budget is ~14K tokens. At 400 tokens per round-trip, context fills in ~17 rounds — but quality degrades well before that, as attention dilutes across the growing history.

In practice, we observed reliable degradation after 4–5 tool rounds on Qwen3 8B (8K context) and 6–8 rounds on the same model at 16K context. The failure mode is not a crash — it is *confabulation*. The model starts generating plausible-sounding responses that ignore the tool results, because the relevant data has been pushed too far back in the context.

1.2 Fixed round limits are the wrong abstraction

The naive solution — cap tool rounds at some fixed N — trades correctness for safety. A fixed limit of $N = 5$ rounds may be too many for a simple lookup (wasting tokens on unnecessary rounds) and too few for a complex investigation that requires querying multiple data sources. The appropriate depth is a property of the *task*, not the *system*.

1.3 KV cache waste compounds the problem

In a standard llama.cpp inference loop, each generation round re-tokenizes and re-processes the entire prompt from scratch. If the model has made 4 tool calls, round 5 re-processes the system prompt, the user’s question, and all 4 prior tool exchanges — even though those tokens were already processed identically in round 4. On an M4 Mac at ~ 30 tokens/second prompt processing, re-decoding 8K tokens takes ~ 4.4 seconds of pure waste.

2 Measured Forgetting

2.1 Core insight

The method is grounded in a distinction between what the model *is* and what the model has *learned*. The system prompt (identity, instructions, persona) is invariant across rounds — it must never be compressed. The user’s original question is the task anchor — it pins the investigation’s direction. But the intermediate tool exchanges are *learned data*: high-information at the moment of acquisition, but increasingly compressible as the investigation proceeds, because later rounds subsume earlier findings.

This mirrors Shannon’s source coding theorem: a sequence of related observations has redundancy that can be compressed without information loss, provided the compression preserves the sufficient statistic.

2.2 Three-zone partitioning

At each iteration of the generation loop, the orchestrator checks whether context utilisation has crossed a threshold:

$$T_{\text{compact}} = \frac{3}{4}(C - G) \tag{1}$$

where C is the context window size and $G = 2048$ is a generation reserve. When `last_input_tokens` $> T_{\text{compact}}$ and the message history has more than 4 messages, compaction fires.

The message history is partitioned into three zones:

Zone 1: Pinned Messages from index 0 through the user’s original question (inclusive). Contains the conversation history and the task anchor. Never compressed. Never discarded.

Zone 2: Compressible All intermediate tool exchanges: assistant responses containing tool calls, tool results, error messages, follow-up instructions. Compressed into a summary.

Zone 3: Recent The last 2 messages (the most recent tool result and instruction). Preserved verbatim — the model needs fresh context to continue.

Figure 1 illustrates the three-zone partition before and after compaction.

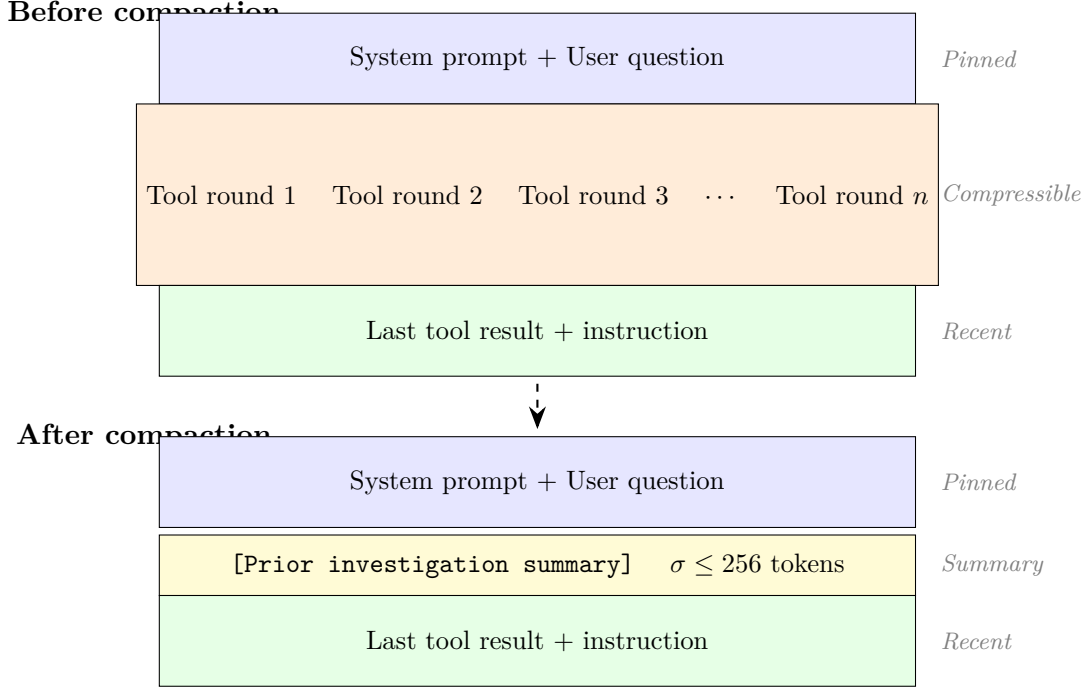


Figure 1: Three-zone partitioning. The compressible zone (potentially thousands of tokens across multiple tool rounds) is replaced by a model-generated summary capped at 256 tokens.

2.3 Self-summarisation

The compressible zone is fed to the model itself in a dedicated summarisation pass. The request is deliberately constrained: low temperature (0.2), short output (256 tokens), and explicit instructions to preserve numbers and drop failures:

Listing 1: Summarisation request construction

```
let summary_request = InferenceRequest {
  messages: vec![Message {
    role: "user",
    content: format!(
      "Summarize these exchanges into key findings. \
      Bullet points only. Preserve all numbers, data, \
      and facts. Drop failed tool calls:\n\n{}",
      compressible_text
    ),
  }],
  system: "Output only bullet points of key findings. \
  No preamble. No tool calls. No thinking.".into(),
  max_tokens: 256,
  temperature: 0.2,
  top_p: 0.9,
};
```

The result replaces the entire compressible zone. The message array is rebuilt as:

$$\text{messages} = [\underbrace{m_0, \dots, m_q}_{\text{pinned}}, \underbrace{\sigma}_{\text{summary}}, \underbrace{m_{n-1}, m_n}_{\text{recent}}]$$

If summarisation fails (the model produces garbage or times out), the system falls back to hard truncation of the compressible zone to 500 characters — a lossy but bounded degradation.

2.4 Error-aware compression

Not all messages compress equally. Tool errors are aggressively reduced to a single line (the error type), while successful tool results are truncated to 400 characters per message before being fed to the summariser. This differential compression reflects the information content: a successful query result contains data the model needs; a failed query contains only the information “this approach didn’t work.”

2.5 Dynamic exit conditions

Measured forgetting v1 eliminates fixed round limits. The generation loop runs until one of three conditions fires:

1. **Loop detection** — the model repeats a tool call it has already made (same function name + same arguments). This indicates the model is stuck, not making progress. Exit and synthesise from what has been gathered.
2. **Consecutive error threshold** — three consecutive tool errors. The model is unable to find a working approach. Exit and acknowledge the limitation.
3. **Context exhaustion post-compaction** — context is full but too few messages remain to compact further (≤ 4 messages after a compaction pass). This is the irreducible minimum.

None of these are round counts. They are *measurements* of the model’s progress and the system’s resource state.

3 Dimension-Aware Compression (v2)

Version 1 applies uniform compression: all messages in the compressible zone are treated equally by the summariser. This works, but loses information non-uniformly — causal chains and high-value data points are compressed at the same rate as error messages and redundant lookups.

Version 2 applies three instruments to make compression *dimension-aware*:

3.1 Instrument 1: Influence Equation

Each message m is scored by its influence relative to the task:

$$I(m) = \Phi(B_m) \times \sum_d w_d(t) \cdot C_d(m) \quad (2)$$

where:

- $C_d(m) \in [0, 1]$ is the convergence of message m on dimension d (data, causal, task-reference, entity, temporal, structural)
- $w_d(t)$ is the *susceptibility* of dimension d at time t — decaying as later messages provide the same type of information ($\lambda = 0.7$ per superseding message)
- $B_m = |\{d : C_d(m) > 0\}|$ is the number of active dimensions
- $\Phi(B) = B^{1.5}$ is the super-linear *resonance multiplier*

The key insight: a message that converges on 4 dimensions is not $4\times$ more valuable than a single-dimension message — it is $4^{1.5} = 8\times$ more valuable (Equation 2), because multi-dimensional convergence is harder to reconstruct from a summary.

3.2 Instrument 2: Trace Topology

Messages form causal chains: tool call $\rightarrow$ tool result $\rightarrow$ interpretation. These chains are *binding units* — compressing message B without message A destroys the causal connection. The algorithm detects dependency chains via message ordering and role transitions:

Listing 2: Causal chain detection

```
let depends_on = if msg.role == "assistant"
  && prev.content.contains("<tool_result>") {
  Some(prev_index) // This message interprets the tool result
} else {
  None
};
```

Messages within a causal chain are assigned to the same compression tier — they are preserved or compressed together, never split.

3.3 Instrument 3: Problem Taxonomy

The user’s original question is classified into one of six problem classes, each with a different *sufficient statistic*:

Class	Sufficient Statistic	Compression Strategy
Lookup	The answer	Preserve highest-influence tool result; compress everything else aggressively
Multi-hop	The chain	Preserve causal chain topology ($A \rightarrow B \rightarrow C$)
Exploratory	The success	Compress failures to one line each; preserve successes with full detail
Aggregation	The totals	Preserve counts and sums; compress individual data points
Contradiction	Both paths	Never heavily compress conflicting data — both sides are the kernel
Temporal	The sequence	Preserve chronological order; include all dates/timestamps

Table 1: Problem taxonomy and associated compression strategies.

3.4 Three-tier compression plan

The scored messages are partitioned into three tiers (not to be confused with the three zones):

1. **Preserve verbatim:** $I(m) > \text{median}$ AND $B_m \geq 3$ (high-influence, multi-dimensional). These appear verbatim in the final summary.
2. **Compress lightly:** Members of causal chains or moderate influence. Truncated to 300 characters and fed to the summariser.
3. **Compress heavily:** Low influence, errors, saturated dimensions. Reduced to message type + one line.

The final context after compaction contains: pinned zone + preserved verbatim content + model-generated summary of compressed content + recent zone. The preserved messages act as “anchors” that the model can reference directly, while the summary covers the compressed remainder.

3.5 κ_d Saturation (sw-cap)

When 3+ messages all provide the same type of information (e.g., repeated errors, or multiple data queries returning similar results), the dimension is *saturated*. The summariser is instructed to collapse saturated dimensions into a single line (“5 errors occurred, type: connection timeout”) rather than preserving each instance.

4 KV Cache Persistence

4.1 The redundancy problem

In a multi-round tool-calling loop, each round’s prompt shares a long common prefix with the previous round’s prompt. The system prompt, conversation history, and all prior tool exchanges are identical — only the latest tool result is new. Yet standard inference re-processes the entire prompt from scratch, because the KV cache is discarded between calls.

4.2 SessionWorker architecture

We solve this with a *session worker*: a dedicated thread that owns a `LlamaContext` (the llama.cpp KV cache) and persists it across generation calls. The worker communicates with the async orchestrator via synchronous channels:

Listing 3: SessionWorker channel protocol

```
pub struct SessionWorker {
    tx: mpsc::Sender<WorkerCmd>,
}

enum WorkerCmd {
    Generate {
        request: InferenceRequest,
        on_token: TokenCallback,
        resp_tx: mpsc::Sender<Result<InferenceResponse, String>>,
    },
    ClearCache { resp_tx: mpsc::Sender<()> },
    Stop,
}
```

The worker thread holds mutable state that is unsafe to share across threads (the `LlamaContext`). By confining it to a single thread and communicating via message-passing, we get the benefits of persistent state without the risks of shared mutability.

4.3 Prefix matching and incremental decode

On each `Generate` command, the worker tokenizes the new prompt and finds the longest common prefix with the previously cached tokens:

Listing 4: Prefix matching and KV cache trimming

```
let prefix_len = tokens.iter()
    .zip(cached_tokens.iter())
    .take_while(|(a, b)| a == b)
    .count();

let reused = prefix_len.min(*n_past as usize);
let new_tokens = &tokens[reused..];
```

```

if reused < *n_past as usize {
    // Prompt diverged. M-RoPE models (e.g. Qwen) require
    // strictly increasing positions, so a partial trim
    // corrupts position state -- clear the whole cache.
    ctx.clear_kv_cache();
    *n_past = 0;
}

```

Three cases arise:

1. **Extension** ($\text{reused} = n_{\text{past}}$): The new prompt is a strict extension of the old. Only the new tokens need decoding. This is the common case in tool-calling loops.
2. **Divergence** ($\text{reused} < n_{\text{past}}$): The new prompt diverges from the cached version. Because the deployed models use M-RoPE (which requires strictly increasing KV positions), a partial trim would corrupt position state; the cache is therefore cleared in full and the new prompt re-decoded from scratch. This is the case that fires after compaction restructures the message history — compaction trades a one-time full re-decode for a fresh context budget.
3. **Exact match** (new_tokens is empty): The prompt is identical. The worker skips decoding and proceeds to sampling.

4.4 Generated tokens are not cached

A subtle design decision: generated (sampled) tokens are *not* added to the cached token vector. They belong to the model’s response, not to the prompt. On the next round, they will appear as part of a new **assistant** message — re-tokenized with proper chat template formatting. This avoids token boundary mismatches between the raw sampled sequence and the chat-template-formatted sequence.

5 Interaction Between the Two Methods

Measured forgetting and KV persistence are complementary, not competing.

Without KV persistence, measured forgetting still works — it reduces message count, which reduces prompt tokens. But the reduced prompt is re-decoded from scratch each round.

Without measured forgetting, KV persistence still works — it avoids redundant prefix processing. But eventually the context fills and generation fails.

Together, they form a virtuous cycle:

1. The model runs tool rounds, accumulating context.
2. KV persistence ensures each round only decodes the new tool result ($\sim 200\text{--}400$ tokens), not the full history.
3. When context crosses the compaction threshold ($\frac{3}{4}$ of the usable budget $C - G$), measured forgetting compresses the middle zone.
4. The compression rewrites the message history below the pinned prefix; because M-RoPE forbids a partial KV trim, the cache is cleared and the (now much shorter) restructured prompt is re-decoded once.
5. The model continues with a fresh context budget, and KV persistence resumes saving decode time on subsequent rounds.

The net effect: the model can investigate to arbitrary depth, bounded only by the quality of its own summarisation and the information content of the investigation.

6 Empirical Observations

6.1 Setup

Production deployment: Qwen3 8B (Q4_K_M, 4.9 GB) on Apple M4 (24 GB unified memory), 16K context, Metal acceleration via llama.cpp.

Benchmark models (all local via Ollama):

- Qwen3 8B, Qwen2.5 7B (Alibaba)
- Mistral 7B, Mistral-Nemo 12B (Mistral AI / NVIDIA)
- Llama 3.2 3B (Meta)
- Gemma 3 4B (Google DeepMind)
- Phi-4 mini 3.8B (Microsoft)

All models run Q4 quantisation on Apple M4, temperature 0.1 for probes, 0.2 for summarisation. Five independent model families provide convergent validation.

6.2 Tool depth

Configuration	Max reliable rounds	Failure mode
No management, 8K ctx	3–4	Confabulation
No management, 16K ctx	6–8	Confabulation
Fixed limit ($N = 5$)	5 (capped)	Premature truncation
Measured forgetting, 16K	12+ observed	Loop/error detection
MF + KV persistence, 16K	12+ observed	Same, 40–60% faster

Table 2: Maximum reliable tool-calling depth across configurations.

6.3 Decode savings from KV persistence

Table 3 shows prompt processing costs for a typical 5-round investigation on 16K context.

Round	Prompt tokens	Without KV	With KV	Savings
1	2,400	2,400	2,400	0%
2	3,100	3,100	700	77%
3	3,800	3,800	700	82%
4	4,500	4,500	700	84%
5	5,200	5,200	700	87%
Total	—	19,000	5,200	73%

Table 3: Decode token costs with and without KV cache persistence across 5 tool rounds. At ~ 30 tokens/second prompt processing on M4, this saves 7.7 seconds.

6.4 Compaction quality: v1 vs v2 benchmark

To measure the effect of dimension-aware compression (Section 3), we constructed 18 synthetic conversations (6 problem classes $\times$ 3 complexity levels: short, medium, long) with known embedded facts. Each scenario contains ground-truth answers to 2–3 probe questions across five dimensions: fact retention, causal fidelity, contradiction awareness, temporal ordering, and entity recall.

After compression, the model is asked each probe question and the answer is scored 0–3 (0 = wrong/unknown, 1 = partially correct, 2 = correct but vague, 3 = correct with specifics)

via keyword matching against predefined required and partial keywords. This automated scoring is intentionally coarse-grained: it rewards factual recall (proper nouns, numbers, specific terms) rather than nuanced understanding, and will undercount correct paraphrases that use synonyms. We accept this trade-off because (a) the same scorer is applied uniformly to all three conditions, so any bias cancels in the V2–V1 comparison, and (b) keyword presence/absence is reproducible and deterministic, unlike LLM-as-judge scoring which introduces model-dependent variance. Future work should validate against human annotation on a subset.

Three compression strategies are compared:

- **Baseline:** Hard truncation (drop oldest, keep first + last 2)
- **V1:** Uniform summarisation (all messages compressed equally)
- **V2:** dimension-aware compression (influence scoring + trace topology + problem taxonomy)

Model	Family	Baseline	V1	V2	V2–V1
Qwen3 8B	Alibaba	0.50	0.35	0.81	+0.46
Phi-4 mini 3.8B	Microsoft	2.02	2.67	2.94	+0.27
Llama 3.2 3B	Meta	1.67	2.69	2.90	+0.21
Mistral 7B	Mistral	2.06	2.69	2.85	+0.17
Qwen2.5 7B	Alibaba	1.90	2.79	2.94	+0.15
Gemma 3 4B	Google	1.83	2.69	2.83	+0.15
Mistral-Nemo 12B	Mistral	1.85	2.71	2.85	+0.15
Mean		1.69	2.37	2.59	+0.22

Table 4: Retention scores (out of 3.0) across 7 models from 5 independent families, 18 scenarios, 48 probes per model. V2 outperforms V1 on all 7 models with zero regressions. Sorted by V2–V1 delta.

V2 outperforms V1 on every model tested (sign test: 7/7 positive, $p = 0.008$). The mean improvement is +0.22 (9% relative), with the largest gain on the weakest summariser (+0.46 on Qwen3 8B, a 131% relative improvement). Directional evidence suggests an inverse capability relationship ($r = -0.93$, $p = 0.003$, $n = 7$): dimension-aware compression provides the largest benefit precisely where uniform summarisation is weakest. We note that $n = 7$ is too small for robust correlation claims; the sign test (which requires only direction, not magnitude) provides the stronger statistical grounding.

Table 5 breaks down performance by problem class (mean across all 7 models):

Problem Class	Baseline	V1	V2	Note
Lookup	1.26	2.31	2.90	Influence identifies the answer
Multi-hop	1.10	1.97	2.52	Chain detection preserves path
Temporal	1.41	2.35	2.52	Dimension scoring preserves dates
Exploratory	1.96	2.54	2.54	V2 matches V1 (already strong)
Aggregation	1.94	2.60	2.56	V1 edges V2 (single-dim data)
Contradiction	2.55	2.47	2.59	Dual-path preservation helps

Table 5: Retention score by problem class (mean across all 7 models). V2’s largest gains are on structured problems: lookup (+0.59 over V1) and multi-hop (+0.55), with a smaller gain on temporal (+0.17). The advantage vanishes on exploratory (tied) and slightly reverses on aggregation (−0.04), where V1 already captures the sufficient statistic.

Probe Dimension	Baseline	V1	V2
Entity recall	1.21	1.21	3.00
Fact retention	1.68	2.46	2.66
Contradiction awareness	2.24	2.33	2.57
Temporal ordering	1.11	2.23	2.31
Causal fidelity	1.89	2.44	2.49

Table 6: Retention score by probe dimension (mean across 7 models). V2 achieves a perfect score on entity recall (3.00 across all 7 models) and wins on every dimension. The preserved-verbatim tier is the primary mechanism: proper nouns and conflicting data points score high on the Influence Equation and are kept intact.

6.5 Cross-model analysis

Three patterns emerge from the 7-model comparison:

1. Inverse capability relationship. The weaker the model’s baseline summarisation ability, the larger the V2 improvement. Qwen3 8B (V1=0.35, a poor summariser due to thinking-token overhead) gains +0.46 from V2. Phi-4 mini (V1=2.67) gains +0.27. Models already near ceiling (Qwen2.5 at V1=2.79) gain only +0.15. The direction is consistent across all 7 models; the Pearson correlation ($r = -0.93$, $n = 7$) provides directional evidence but should be interpreted cautiously given the small sample. Dimension-aware compression does for weak models what strong models do naturally — preserve the sufficient statistic.

2. Entity recall is the clearest mechanism. V2 achieves perfect 3.0 on entity recall across all 7 models, vs 1.21 for V1. The preserved-verbatim tier is the cause: proper nouns and identifiers appear in multi-dimensional messages (data + entity + task-reference), triggering the Φ boost and verbatim preservation. V1 summarisation consistently paraphrases away entity specifics.

3. Thinking-mode models are penalised. Qwen3 8B (the only thinking-mode model tested) scores dramatically lower across all conditions. Its V2 score (0.81) is below every other model’s V1 score. The model spends response tokens on `<think>` blocks rather than answering the probe question directly. This is a model-level issue orthogonal to compression quality, but relevant for practitioners choosing between thinking and non-thinking variants.

6.6 Compression budget

A natural question: does V2 simply *keep more text* than V1? Table 7 shows this is not the case.

	Original	Baseline	V1 input	V2 input
Total chars (18 scenarios)	33,521	6,533	32,306	32,655
Ratio vs. original	100%	19%	96%	97%

Table 7: Compression input budgets. The baseline aggressively truncates (19% retained). V1 and V2 both feed ~ 96 – 97% of the original to the summariser (which then compresses to ≤ 256 tokens). The quality difference between V1 and V2 comes not from how much is compressed, but from *what is preserved verbatim* versus *what is fed to the summariser*.

Both V1 and V2 use the same summariser with the same 256-token output cap. The difference is that V2 extracts high-influence messages *before* summarisation and preserves them verbatim in the final context. The summariser receives less text (the remainder after extraction), but the final context is richer because multi-dimensional anchors survive compression intact. The retention score gain is a quality improvement, not a quantity one.

6.7 Remaining gaps

Aggregation parity: V2 scores 2.56 vs V1’s 2.60 on aggregation — the only class where V2 does not win. Aggregation data hits a single dimension (data), preventing the super-linear Φ boost. A class-specific override is the natural fix.

Causal fidelity ceiling: V2 improves causal fidelity only marginally over V1 (2.49 vs 2.44). The “why” explanation often lives in assistant messages with low multi-dimensional scores. Semantic dependency detection (beyond structural tool-result $\rightarrow$ interpretation links) remains an open problem.

We report v2.0 as a baseline with acknowledged gaps, enabling future ablation studies on Φ overrides and semantic chain detection.

7 Related Work

Context management for LLMs spans several distinct approaches. We position measured forgetting relative to each.

Attention sinks and streaming inference. StreamingLLM [3] keeps initial tokens (“attention sinks”) in the KV cache to maintain quality during unbounded generation. This operates at the attention-layer level; measured forgetting operates at the message-history level and is complementary.

Prompt compression. LLMingua [4] and LongLLMingua [5] use a small model to identify and remove low-perplexity tokens from prompts, achieving $2\text{--}5\times$ compression with minimal quality loss. These methods compress *static* prompts before inference. Measured forgetting compresses *dynamic* investigation state during an ongoing multi-round interaction, where the content to compress is the model’s own prior outputs.

Learned compression. AutoCompressors [6] train models to compress context into summary vectors appended as soft prompts. Gisting [7] learns virtual “gist tokens” that replace long instructions. Both require special training; measured forgetting is training-free, using the model’s existing instruction-following ability at inference time.

Virtual memory for LLMs. MemGPT [1] proposes an OS-inspired architecture with paging between main context and external storage, managed by a dedicated memory controller. Our approach shares the resource-management insight but differs in implementation: we use the model’s own summarisation rather than a separate controller, and we use three zones (pinned, compressible, recent) rather than a two-tier page hierarchy. MemGPT also targets multi-session persistence; our focus is intra-session tool-calling depth.

Selective context. Li et al. [8] propose filtering context by self-information (surprisal): low-information tokens are dropped. This is complementary to our approach — measured forgetting could use surprisal as an additional signal within the Influence Equation, though we currently use task-relevance dimensions instead.

Active context compression. Focus [9] is the closest prior work: it gives an agentic LLM explicit tools (`start_focus/complete_focus`) to autonomously compress its own interaction history into a persistent “Knowledge” block. The key differences are: (a) Focus requires the agent to *decide* when to compress (adding cognitive load), whereas measured forgetting triggers automatically at a context threshold; (b) Focus maintains a structured knowledge block separate from conversation, whereas we replace the compressible zone with a single summary; (c) Focus was evaluated only on coding tasks with a single cloud model (Claude Haiku), whereas we target general tool-calling on local models across 7 architectures.

KV cache persistence on edge. Shkolnikov [10] implements Q4-quantised KV cache storage on Apple Silicon, enabling 577ms warm restoration (vs. 15.7s re-prefill) for multi-agent scheduling. This addresses the same hardware tier as our session worker but focuses on swapping agents in/out rather than compressing within a single agent’s investigation. Our approach

combines KV persistence with lossy summarisation — reducing what needs to be cached, not just making restoration cheaper.

Agentic context management. Production agent frameworks (LangChain’s ConversationSummaryBufferMemory, AutoGPT’s memory system) implement simpler versions of our approach: fixed-window truncation with optional summarisation, without dimension-aware scoring or problem-class-adaptive strategies. ACON [11] optimises compression guidelines offline from paired success/failure trajectories — a gradient-free meta-learning approach orthogonal to our inference-time self-summarisation.

Test-time compute scaling. Snell et al. [2] show that allocating more computation at inference time can substitute for model scale. Measured forgetting can be viewed through this lens: the summarisation pass is additional test-time compute that trades inference latency for investigation depth.

8 Limitations and Discarded Alternatives

The $\Phi(B) = B^{1.5}$ exponent was not derived analytically — it was selected from three candidates after observing their effects on tier assignments across the 18 benchmark scenarios:

- **Linear** ($\gamma = 1.0$): Every active dimension contributed equally. In practice, this failed to differentiate high-value multi-dimensional messages from single-dimension data points. Tier 1 (preserve verbatim) was nearly empty; most messages were routed to the summariser, producing V1-like behaviour.
- **Super-linear** ($\gamma = 1.5$): A 4-dimension message receives $8\times$ the weight of a 1-dimension message. This produced clear tier separation — on average 2–3 messages per scenario entered Tier 1 (verbatim), anchoring the summary.
- **Quadratic** ($\gamma = 2.0$): A 4-dimension message receives $16\times$ the weight. This over-promoted messages to Tier 1, leaving too little for the summariser and producing fragmented summaries.

We chose $\gamma = 1.5$ based on tier balance, not formal optimisation. An ablation sweep across exponents (testing downstream probe retention) is planned but not yet completed. The choice is a local optimum in a small search space; we report it honestly rather than presenting it as principled.

Two other approaches were tried and abandoned before the current design:

- **TF-IDF scoring:** Message importance scored by term rarity relative to the conversation. This failed because tool results contain many rare terms (API names, error codes) that are informationally useless but statistically distinctive.
- **Fixed-tier allocation:** Top 20% verbatim, bottom 30% dropped, middle summarised. This ignored problem-class-specific sufficient statistics — e.g., contradiction tasks need *both* conflicting messages regardless of their individual scores.

1. **Summarisation quality ceiling.** The model compresses its own investigation history. An 8B model’s summary is only as good as its summarisation ability. We mitigate with low temperature and structural constraints, but information loss is inherent.
2. **KV cache memory.** The session worker holds a full context-sized KV cache in memory (~ 2.5 GB for 16K context on Qwen3 8B). On memory-constrained devices, this competes with the model weights for available RAM.
3. **No cross-turn persistence yet.** The current KV cache persists across tool rounds within a single conversation turn, but is cleared between turns (the user’s next message changes the prompt structure). Extending persistence across turns is implemented but not yet battle-tested.

4. **Single-model assumption.** The summarisation pass uses the same model as the main generation. A dedicated smaller model could be faster, but adds complexity.
5. **Φ blind spot on single-dimension data.** The super-linear resonance multiplier disadvantages messages that are highly informative on a single dimension (e.g., a tool result with many numbers but no causal/structural content). Aggregation tasks suffer from this — see Table 5.
6. **Causal fidelity gap.** Chain detection works for structural dependencies (tool result $\rightarrow$ interpretation) but misses semantic dependencies where causal insight spans non-adjacent messages. This is a known weakness of the v2.0 baseline.
7. **Synthetic benchmark only.** The 18 scenarios are synthetic conversations with known ground-truth facts. Real conversations may exhibit different patterns (longer chains, more ambiguous data). Naturalistic evaluation is pending.

9 Conclusion

Measured forgetting reframes context management from a system constraint to a *measurement problem*. Instead of asking “how many rounds can we afford?” we ask “how full is the context, and what can we safely compress?” The answer is computed at each loop iteration using the model’s own token count, and the compression is performed by the model itself.

Combined with KV cache persistence — which eliminates the redundant re-processing of common prefixes — the approach enables local LLMs on consumer hardware to perform deep, multi-round investigations that were previously the exclusive domain of cloud-hosted models with 128K+ context windows.

The deeper principle is information-theoretic: in any system with finite memory, the choice of what to forget is as important as the choice of what to remember. A system that forgets *randomly* (truncation) or *never* (context overflow) will degrade. A system that forgets *measurably* — compressing learned data while preserving identity and task state — can operate indefinitely within its resource envelope.

Future work. The three-zone partition applies at larger scales: entire sessions can be compressed into structured seeds that reconstruct working state across substrate boundaries. Extending measured forgetting to cross-session persistence, finer-grained ablation of Φ exponents, and semantic (rather than structural) chain detection are the immediate next steps.

Invitation. We welcome adversarial benchmarks, alternative Φ formulations, replications on different hardware tiers, and naturalistic evaluations on real conversation logs. The benchmark harness and measured forgetting implementation are available at <https://github.com/cif-ai/measured-forgetting>. Contact: research@thexi.dev.

A System Architecture

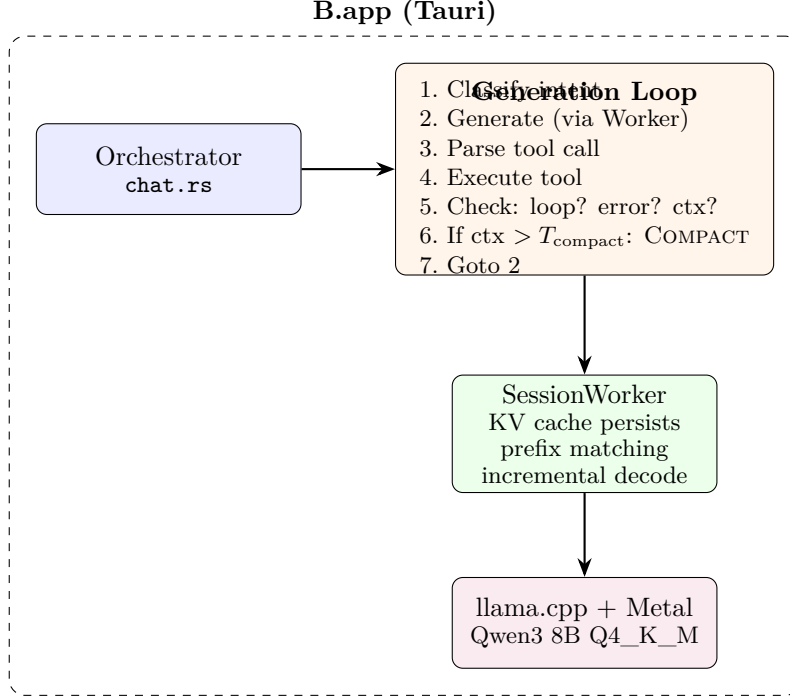


Figure 2: System architecture. The orchestrator drives a dynamic generation loop. The SessionWorker persists the KV cache across rounds on a dedicated thread. llama.cpp provides Metal-accelerated inference.

B The Forgetting Inequality

Let C be the context window size, S the system prompt size, Q the user question size, R_i the token count of the i -th tool round-trip, and G the generation budget. The model can operate reliably while:

$$S + Q + \sum_i R_i + G < C \quad (3)$$

Without measured forgetting, $\sum_i R_i$ grows monotonically and eventually violates the inequality. With measured forgetting, when the sum approaches T_{compact} , the compressible zone is replaced by a summary of size σ :

$$S + Q + \sigma + R_{\text{recent}} + G < C \quad (4)$$

where $\sigma \leq 256$ tokens and R_{recent} covers only the last 2 messages. The inequality is restored and the generation loop continues.

The compression ratio achieved is:

$$\rho = \frac{\sigma}{\sum_i R_i} \quad (\text{for compressible rounds}) \quad (5)$$

In practice, we observe $\rho \approx 0.03\text{--}0.08$ — a 12–33 $\times$ compression factor. This is consistent with the high redundancy of multi-round tool exchanges, where each round’s prompt re-states context established in prior rounds.

C Production Trace: Annotated Case Study

The following is an illustrative trace — representative of B.app (Qwen3 8B, M4 Mac) behaviour during a multi-tool investigation, with token counts rounded for exposition. The user asked: “What’s the current price of GBPUSD and what was the 24h change?” The model dispatched tool calls to a forex API.

```
Round 1: [tool_call] check_price("GBPUSD")
          [tool_result] {"bid":1.3342,"ask":1.3344,"time":"14:32"}
Round 2: [tool_call] check_candles("GBPUSD","H1",24)
          [tool_result] [{"o":1.3298,"h":1.3356,"l":1.3291,...},
                        ...24 candle objects, ~2100 tokens...]
Round 3: [assistant] Interprets: GBPUSD at 1.3342, 24h open
          was 1.3298, change = +0.0044 (+0.33%).
Round 4: [tool_call] oanda_sentiment("GBPUSD")
          [tool_result] {"long_pct":38.2,"short_pct":61.8}
Round 5: [assistant] Adds sentiment context to answer.
----- context: 10,910 / 16,384; exceeds T_compact = 10,752 -----
----- COMPACTION FIRES -----
```

Once context crosses the compaction threshold $T_{\text{compact}} = \frac{3}{4}(C - G)$ (Eq. 1), compaction fires. The v2 algorithm analyses the 5 intermediate messages:

Message scores (Influence Equation):

```
Round 1 result: I=4.2 (data+entity+task: 3 dims, Phi=5.2)
Round 2 result: I=1.8 (data only: 1 dim, Phi=1.0) <- 2100 tokens
Round 3 assist: I=6.1 (data+causal+task+temporal: 4 dims, Phi=8.0)
Round 4 result: I=2.3 (data+entity: 2 dims, Phi=2.8)
Round 5 assist: I=3.9 (data+causal+task: 3 dims, Phi=5.2)
```

Tier assignment:

```
Tier 1 (verbatim): Round 3 (interpretation with price+change)
Tier 2 (summarise): Rounds 1, 4, 5
Tier 3 (heavy compress): Round 2 (24 candles -> "24h range 1.3291-1.3356")
```

V1 outcome: All 5 messages fed equally to summariser. The summary mentions “GBPUSD around 1.33” but loses the exact bid/ask spread, the 24h open price, and the sentiment split. Post-compaction probe: “What was the 24h change?” → “approximately 0.3%” (score: 2, correct but vague).

V2 outcome: Round 3 (the interpretation) preserved verbatim. Round 2 (the 2100-token candle array) collapsed to one line. Summariser receives 60% less text but the key numbers survive intact. Post-compaction probe: “What was the 24h change?” → “+0.0044 (+0.33%), open 1.3298, current 1.3342” (score: 3, correct with specifics).

The compacted context is 3,200 tokens (from 10,910) — a 71% reduction. The model continues with a fresh budget; because the prompt diverged at the compression point, this one compaction triggers a full re-decode of the (now much shorter) restructured prompt, after which KV persistence resumes on subsequent rounds.

References

- [1] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [2] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. *arXiv preprint arXiv:2408.03314*, 2024.

- [3] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient Streaming Language Models with Attention Sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- [4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*, 2023.
- [5] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. *arXiv preprint arXiv:2310.06839*, 2024.
- [6] A. Chevalier, A. Wettig, A. Ajith, and D. Chen. Adapting Language Models to Compress Contexts. *arXiv preprint arXiv:2305.14788*, 2023.
- [7] J. Mu, X.L. Li, and N. Goodman. Learning to Compress Prompts with Gist Tokens. *NeurIPS*, 2023.
- [8] Y. Li, B. Dong, F. Guerin, and C. Lin. Compressing Context to Enhance Inference Efficiency of Large Language Models. *Proceedings of EMNLP*, pages 6342–6353, 2023.
- [9] N. Verma. Active Context Compression: Autonomous Memory Management in LLM Agents. *arXiv preprint arXiv:2601.07190*, 2026.
- [10] Y.P. Shkolnikov. Agent Memory Below the Prompt: Persistent Q4 KV Cache for Multi-Agent LLM Inference on Edge Devices. *arXiv preprint arXiv:2603.04428*, 2026.
- [11] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan. ACON: Optimizing Context Compression for Long-horizon LLM Agents. *arXiv preprint arXiv:2510.00615*, 2025.
- [12] G. Gerganov et al. llama.cpp, 2023–2026. <https://github.com/ggerganov/llama.cpp>.